
Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Introduction to Boolean Algebra & Logic Operators

History – Boolean Algebra



Boolean Algebra – George Boole
(1854, A British mathematician)



The branch of algebra in which the values of the variables are the truth values true and false.

Boolean Algebra

- There are only two states
 - State 0
 - State 1
- Usually, a variable is considered in state 1 unless it is specifically designated with a negation.
 - x – variable x is assumed to be in state 1
 - $\bar{x}$ – it is assumed that $\bar{x}$ denotes state 0

Basic Operations: Boolean Algebra

$A.B$

AND

$A+B$

OR

$\bar{A}$

NOT

Boolean Function

- A Boolean function is an expression that consists of binary variables and boolean logic operations.

$$f(x, y) = x \cdot \bar{y} + \bar{x} \cdot (x + y)$$

- A Boolean function can be evaluated for all possible values of the variables.
- Finite number of possible value combinations for all the variables.

Truth Table

- A boolean function can be expressed in a truth table.
- The number of rows will be 2^n where n is the number of variables in the function.

$$F = x + y.\bar{z}$$

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth Table (Cont.)

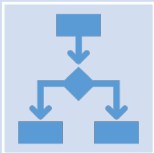
- Truth table can be elaborated with additional columns for each term



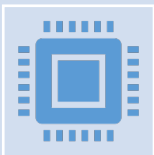
x	y	z	$y.\bar{z}$	F
0	0	0	0	0
0	0	1	0	0
0	1	0	1	1
0	1	1	0	0
1	0	0	0	1
1	0	1	0	1
1	1	0	1	1
1	1	1	0	1

$$F = x + y.\bar{z}$$

Algebra to Logic Circuit



The operational behavior of a Boolean algebraic expression can be represented with a circuit diagram composed of logic gates.



A logic gate is a piece of hardware which emulates Boolean algebraic operation(s) and operates on Boolean inputs.

Inside of a Logic Gate

- Transistors
- Resistors
- Diodes

Transistor AND Gate

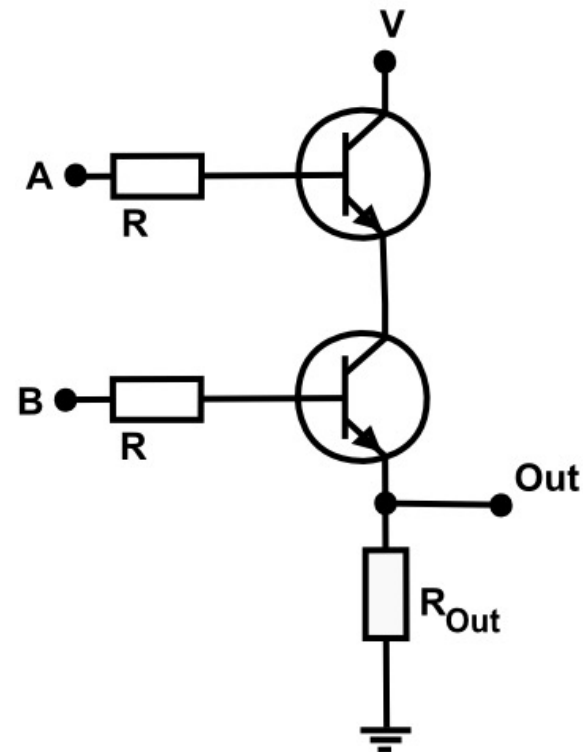
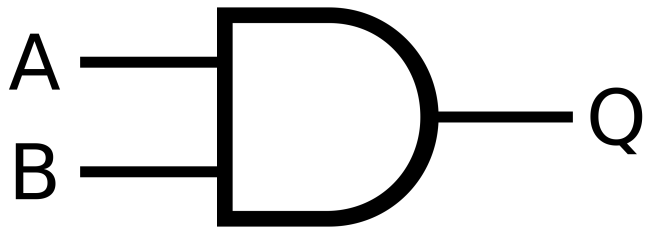


Image Source - https://en.wikipedia.org/wiki/AND_gate

AND Operation

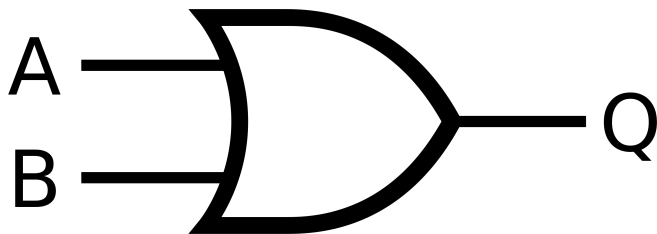
- Operator is symbolically denoted as follows:
 - $A \cdot B$
 - $A \wedge B$



A	B	$Q = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

OR Operation

- Operator is symbolically denoted as follows:
 - $A + B$
 - $A \vee B$



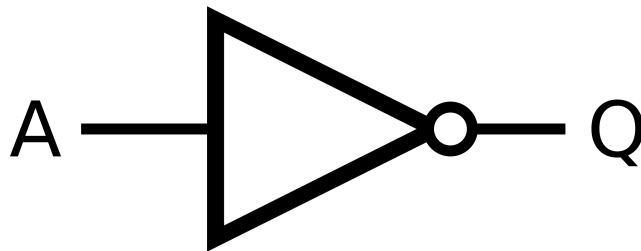
A	B	$Q = A+B$
0	0	0
0	1	1
1	0	1
1	1	1

NOT Operation

- Operator is symbolically denoted as follows:

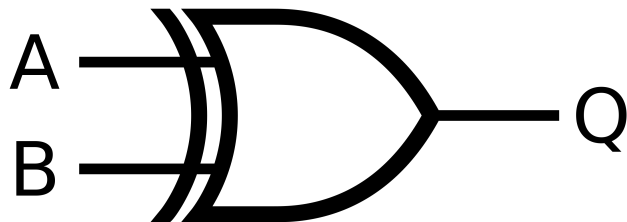
- $\bar{A}$
- $\neg A$
- $\sim A$
- A'

A	$Q = \bar{A}$
0	1
1	0



XOR Operation

- Operator is symbolically denoted as follows:
 - $A \oplus B$

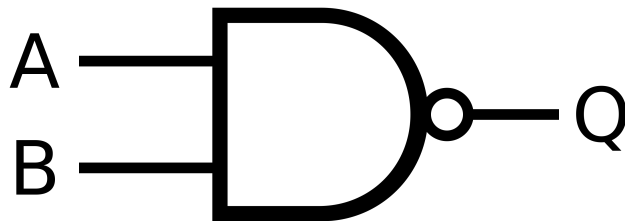


A	B	$Q = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

NAND Operation

- Operator is symbolically denoted as follows:

- $\overline{(A \cdot B)}$
- $\overline{(A \wedge B)}$
- $(A \cdot B)'$
- $\neg(A \wedge B)$



A	B	$\overline{(A \cdot B)}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR Operation

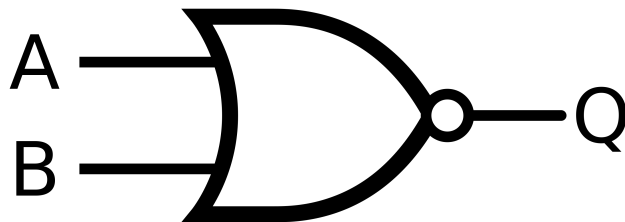
- Operator is symbolically denoted as follows:

- $\overline{(A + B)}$

- $\overline{(A \vee B)}$

- $(A + B)'$

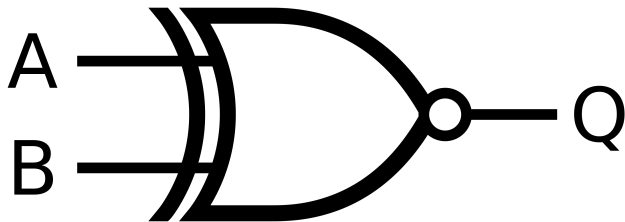
- $\neg(A \vee B)$



A	B	$\overline{(A + B)}$
0	0	1
0	1	0
1	0	0
1	1	0

XNOR Operation

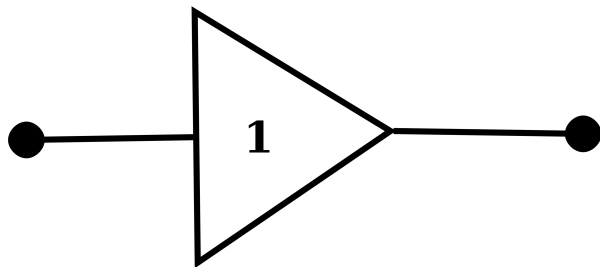
- Operator is symbolically denoted as follows:
 - $\overline{A \oplus B}$



A	B	$\overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1

Buffer

- A basic logic gate that passes its input, unchanged, to its output.
- To increase the propagation delay of circuits



A	Q
0	0
1	1

Logic Gates Manufacturing

- Gates are built into IC form
- Single IC may have several gates
- Typically one type of gates in an IC
- Multi input single gate ICs available

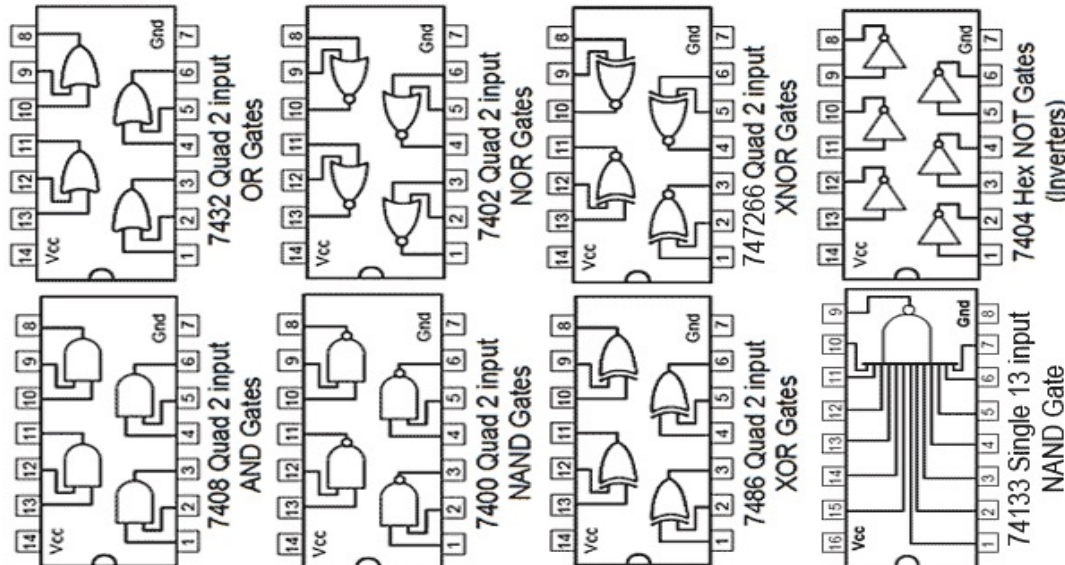


Image Source <https://learnabout-electronics.org/Digital/dig21.php>

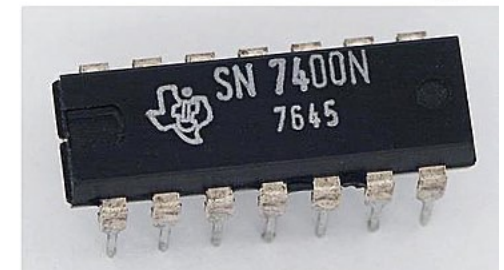
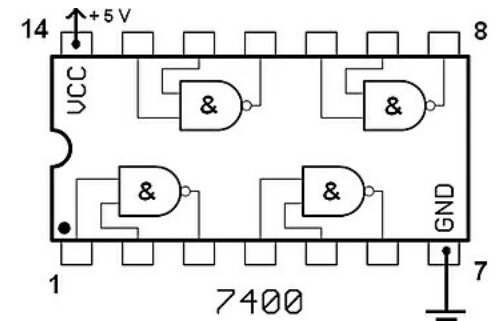
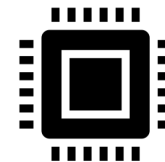


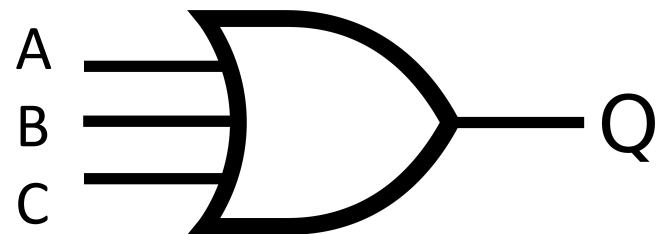
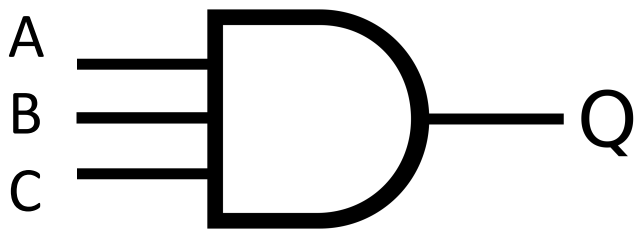
Image Source

https://en.wikipedia.org/wiki/Logic_gate



Multi Input Logic Gates

- Gates with more than two inputs are available
- AND Gate with three inputs (A.B.C)
- OR Gate with ~~four~~^{three} inputs (A+B+C)



Multi Input Logic Gates (Cont.)

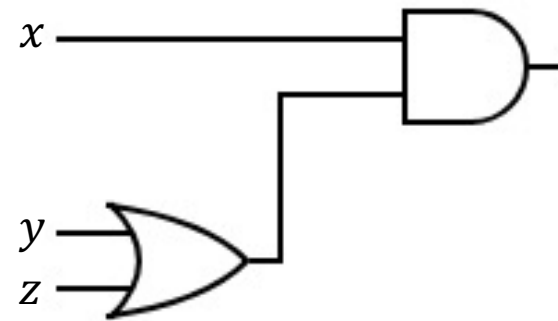
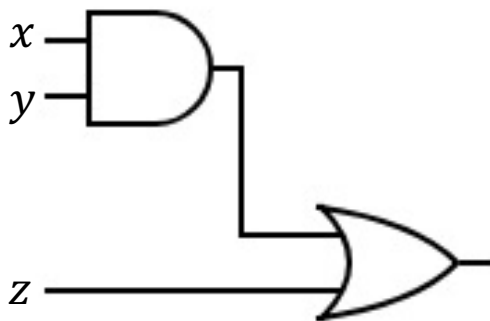
- Gates with more than two inputs are available
- AND Gate with three inputs
 - $F = x \wedge y \wedge z$
- OR Gate with three inputs
 - $F = x \vee y \vee z$
- XOR Gate with three inputs
 - $F = x \oplus y \oplus z$



Drawing Logic Circuits

- How to draw the logic circuit for a given boolean expression?

$$F = x \cdot y + z$$



Operator Precedence



Parentheses



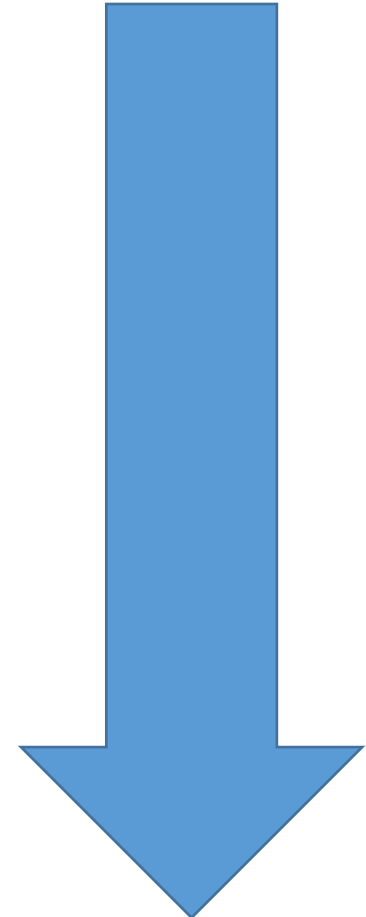
NOT



AND



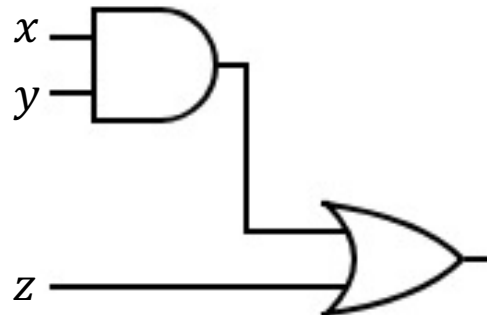
OR



Drawing Logic Circuits

- The logic circuit for a given boolean expression can be drawn using appropriate gates.

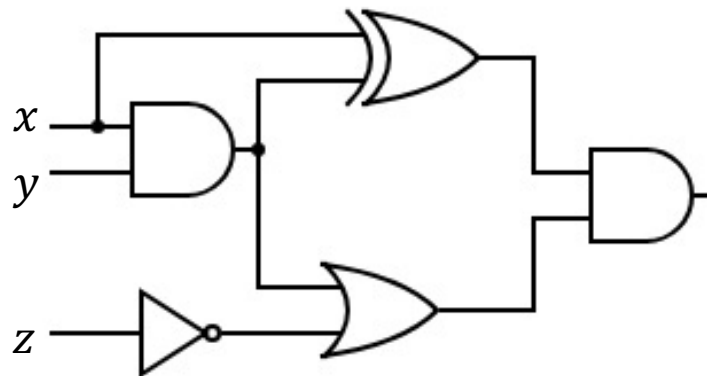
$$F = x.y + z$$



Drawing Logic Circuits

- Complex expressions require complex gate arrangements.

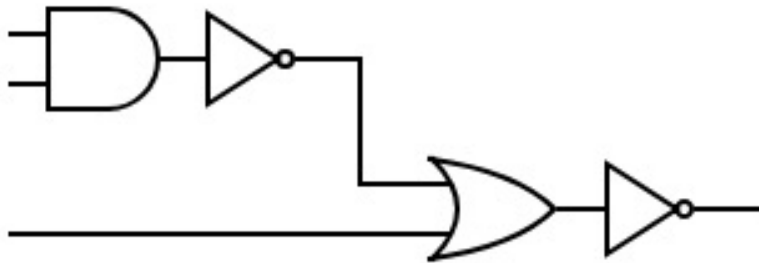
$$F = (x \oplus (x \cdot y)) \cdot (x \cdot y + \bar{z})$$



Drawing Logic Circuits (Cont.)

- Multiple circuits for the same expression

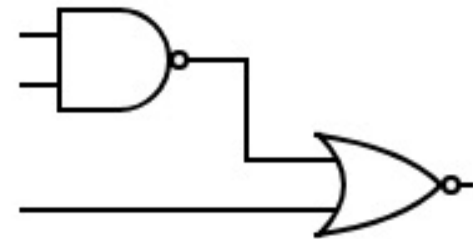
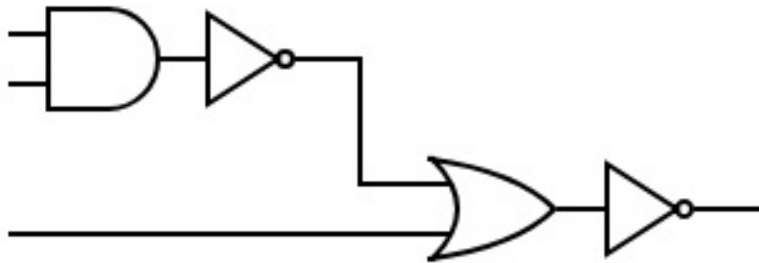
$$F = \overline{\overline{x \cdot y} + z}$$



Drawing Logic Circuits (Cont.)

- Multiple circuits for the same expression

$$F = \overline{\overline{x \cdot y} + z}$$



Thank You..!

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Forms of Boolean Algebraic Expressions

Boolean Expressions

- There can be multiple boolean expressions for a single operational behavior (single truth table).
- Thus, there can be multiple circuit structures with different logic gate arrangements for the same behavior.
- Simple and minimal circuit structure is always preferred.
 - Efficiency
 - Cost

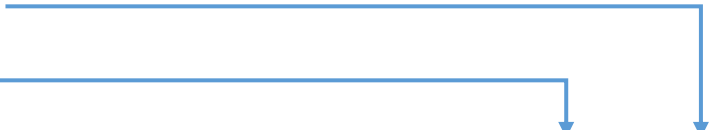
Simple Expression

- Boolean algebra helps to simplify expressions.

- $F1 = \bar{x}\bar{y}z + \bar{x}yz + x\bar{y}$

- $F2 = x\bar{y} + \bar{x}z$

- You will get two circuits if you draw circuits for F1 and F2.



x	y	z	$F2$	$F1$
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	0	0

Expression to Truth Table

- When a boolean expression is given, we can formulate the truth table for the expression.
- Identify the variables in the expression
 - If there are n variables, then there will be 2^n combinations of values
 - 2^n rows in the truth table
- Derive the resulting value for each value combination in the truth table rows.

Expression to Truth Table. (Cont.)

- Ex.

$$F(x, y, z) = x.y.\bar{z} + \bar{y}.z$$

- Three variables in $F(x, y, z)$
- There will be 2^3 combinations
- There will be 2^3 rows in the truth table

Expression to Truth Table. (Cont.)

- Ex.

$$F(x, y, z) = x \cdot y \cdot \bar{z} + \bar{y} \cdot z$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Truth Table to Expression

- When a truth table is given, we can formulate the boolean algebraic expression to represent it.
- There are two forms (*Canonical* forms) in presenting the expression.
 - Sum of *Minterms* (Standard Products)
 - Product of *Maxterms* (Standard Sums)

Minterm

- In a three variable boolean expression, we can combine 3 variables with an AND operator.
- There are 8 possible such arrangements.
 - $\bar{x}.\bar{y}.\bar{z}$
 - $\bar{x}.\bar{y}.z$
 -
 - $x.y.z$
- Each of these AND terms is called ***Minterm*** (or Standard Product).
- E.g. - $(x.y)$ is **not** a ***Minterm*** in this context.

Maxterm

- In a three variable boolean expression, we can combine 3 variables with an OR operator.
- There are 8 possible such arrangements.
 - $\bar{x} + \bar{y} + \bar{z}$
 - $\bar{x} + \bar{y} + z$
 -
 - $x + y + z$
- Each of these OR terms is called **Maxterm** (or Standard Sum)
- E.g. - $(\bar{y} + z)$ is not a **Maxterm** in this context.

Thank You..

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Canonical Forms & Standard Forms

[Sum – Product] in Mathematics



A **product** is the result of multiplying, or an expression that identifies factors to be multiplied.



A **sum** is the aggregate of two or more numbers or particulars as determined by or as if by the mathematical process of addition.

Sum of Minterms

- Disjunction of terms where each term is a conjunction of literals.
- The OR operations are performed on the terms that are made by AND operations.
- Ex.

$$F = \bar{x}.y.\bar{z} + x.\bar{y}.\bar{z} + x.\bar{y}.z + x.y.\bar{z} + x.y.z$$

- Each term in the expression is referred as ***Minterm*** or ***Standard Product***.

Product of Maxterms

- Conjunction of terms where each term is a disjunction of literals.
- The AND operations are performed on the terms that are made by OR operations.
- Ex.

$$F = (x + y + z). (x + y + \bar{z}). (x + \bar{y} + \bar{z})$$

- Each term in the expression is referred as ***Maxterm*** or ***Standard Sum***.

Standard Forms

- In ***Canonical*** Form, each term must contain all the variables in the truth table.
- But ***Standard Form*** is an alternative way to express Boolean function.
- A term in an expression that is expressed in a standard form doesn't need to contain all the variables in the truth table.
- There are two standard forms.
 - Sum of Products
 - Product of Sums

Standard Form – Sum of Products

- OR operation is performed over several terms where each term is made up by performing AND operation over several literals.

- Ex.

$$F = \bar{x}.y + \bar{y}.\bar{z} + x.\bar{y}.z + y$$



$$F = \bar{x}.y + \bar{y}.(x + \bar{z}) + x.\bar{y}.z + y$$



Standard Form - Product of Sums

- AND operation is performed over several terms where each term is made up by performing OR operation over several literals.

- Ex.

$$F = y \cdot (x + z) \cdot (\bar{x} + \bar{y} + \bar{z})$$



$$F = y \cdot (x + z \cdot \bar{y}) \cdot (\bar{x} + \bar{y} + \bar{z})$$



Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Truth Table to Canonical Forms

A Row of a Truth Table

- A row of a truth table represents a possible state of the expression.
- The expression cannot take two states simultaneously.
- Ex. Consider the 3rd row
 - It says the expressions results '1' when $x = 0$ and $y = 1$ and $z = 0$
 - Note the Conjunction (AND)
 - Thus, it can be formalized in to $\bar{x}.y.\bar{z}$

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth Table to Expression

- All such rows that result '1' say the expression will be '1' in either case
- The expression will be '1' when the variables are set as rows 3 **or** 5 **or** 6 **or** 7 **or** 8.

- Note the Disjunction (OR)
- Thus, it can be formalized as follows:

$$F = \bar{x}.y.\bar{z} + x.\bar{y}.\bar{z} + x.\bar{y}.z + x.y.\bar{z} + x.y.z$$

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Expression in “**Sum of Minterms**” Form

Terms in Disjunctive Form

- If we take the 3rd row, we understood that it represents $F_3 = \bar{x} \cdot y \cdot \bar{z}$

- When $x = 0$ and $y = 1$ and $z = 0$, F_3 results 1

- If one of $\bar{x}$ or y or $\bar{z}$ becomes 0, then F_3 will result 0.

- $\bar{x} = 0$ OR $y = 0$ OR $\bar{z} = 0$ will result the complement of F_3

- Therefore, complement of F_3 can be written in disjunctive form.

- $\overline{F_3} = x + \bar{y} + z$

- $\therefore F_3 = \overline{(x + \bar{y} + z)}$

De Morgan's Theorem

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

De Morgan's Theorem

- We have proved that,

$$\bar{x} \cdot y \cdot \bar{z} = \overline{(x + \bar{y} + z)}$$

- Similarly, we can prove that,

$$\overline{(x \cdot \bar{y} \cdot z)} = \bar{x} + y + \bar{z}$$

- This is a theorem in Boolean Algebra and applicable for any number of variables.

Expression in Conjunctive Form

- We derived the expression of the truth table as follows:

$$F = \bar{x}.y.\bar{z} + x.\bar{y}.\bar{z} + x.\bar{y}.z + x.y.\bar{z} + x.y.z$$

- This means when at least one of those terms become 1 then F will result 1.

- It implies when all the terms in F are **negated** at the same time, it will result 0.

- $\overline{(\bar{x}.y.\bar{z})}.(\overline{x.\bar{y}.\bar{z}}).\overline{(x.\bar{y}.z)}.\overline{(x.y.\bar{z})}.\overline{(x.y.z)}} = \bar{F}$

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

PoM - Representation

- Similarly, $\bar{F}$ can be written as $\bar{F} = \bar{x}.\bar{y}.\bar{z} + \bar{x}.\bar{y}.z + \bar{x}.y.z$
- When all the term in $\bar{F}$ are negated, then it will result $\bar{\bar{F}}$
- $\bar{\bar{F}} = \overline{(\bar{x}.\bar{y}.\bar{z})}.\overline{(\bar{x}.\bar{y}.z)}.\overline{(\bar{x}.y.z)}$
- As we studied in the slide “[De Morgan’s Theorem](#)”, each term in $\bar{\bar{F}}$ can be written in disjunctive form.
- $\bar{\bar{F}} = (x + y + z).(x + y + \bar{z}).(x + \bar{y} + \bar{z})$
where $\bar{\bar{F}} = F$
- $F = (x + y + z).(x + y + \bar{z}).(x + \bar{y} + \bar{z})$

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Expression in “**Product of Maxterms**” Form

Truth Table to Expression - Notes

- For a given truth table, there can be a logical expression in two canonical forms,
 - Sum of Minterms
 - Product of Maxterms
- Conversion of expressions in SoM to PoM and vice versa can be done.

Thank You..

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Canonical Forms to Truth Table

SoM Expression to Truth Table

- Each Minterm is associated with a row
 - Find the row corresponds to each Minterm
 - Fill the result column with '1'
- Fill all the result column for all the remaining rows with '0'
- Ex. $F = \bar{x}.\bar{y} + x\bar{y} + x.y$

x	y	F
0	0	1
0	1	0
1	0	1
1	1	1

PoM Expression to Truth Table

- **Intuition:** If a single Maxterm becomes zero then the entire expression would result zero.
- Literals in a Maxterm are combined with OR operation. Ex. $(x + y + z)$.
- All literals in a term are inverted and combined with AND operation $(\bar{x} \cdot \bar{y} \cdot \bar{z})$, would give the logical complement of the same term (De Morgan's Thrm).
- Then we can map each product term to a row in the truth table.

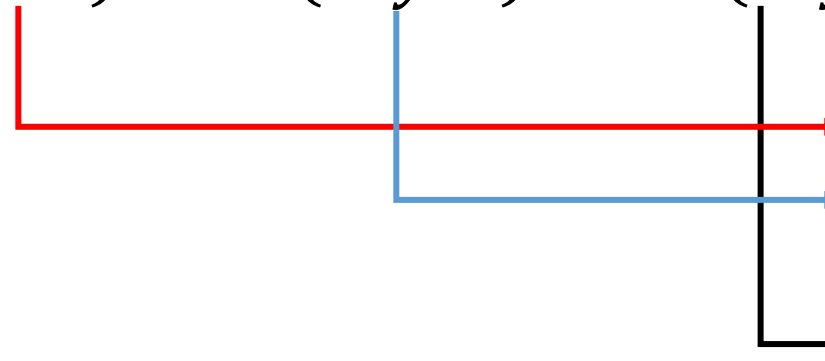
PoM Expression to Truth Table (Cont.)

- Negate each literal in a Maxterm.
- Assuming logical AND over all literals, select the row.
 - Fill the result column with '0'
- Fill the result column for all the remaining rows with '1'.

PoS Expression to Truth Table (Ex.)

- $F = (x + y + z). (x + y + \bar{z}). (x + \bar{y} + \bar{z})$

- $\bar{F} = (\bar{x}.\bar{y}.\bar{z}) + (\bar{x}.\bar{y}.z) + (\bar{x}.y.z)$



x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Exercises

- Derive the logical expression for the given truth table in
 - Sum of Minterm form
 - Product of Maxterms form

x	y	z	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Exercise – Sum of Minterms

- Sum of Minterms

$$F = \bar{x}.\bar{y}.\bar{z} + \bar{x}.\bar{y}.z + \bar{x}.y.\bar{z} + x.\bar{y}.\bar{z} + x.y.\bar{z} + x.y.z$$

$$F(x, y, z) = \sum (0, 1, 2, 4, 6, 7)$$

Note that row index starts from 0

	x	y	z	F
$\bar{x}.\bar{y}.\bar{z}$	0	0	0	1
$\bar{x}.\bar{y}.z$	0	0	1	1
$\bar{x}.y.\bar{z}$	0	1	0	1
	0	1	1	0
$x.\bar{y}.\bar{z}$	1	0	0	1
	1	0	1	0
$x.y.\bar{z}$	1	1	0	1
$x.y.z$	1	1	1	1

Exercise – Product of Maxterms

- Product of Maxterms

$$G = (x + \bar{y} + \bar{z}).(\bar{x} + y + \bar{z})$$

$$G(x, y, z) = \prod(3,5)$$

Note that row index starts from 0

x	y	z	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

$$x + \bar{y} + \bar{z}$$

$$\bar{x} + y + \bar{z}$$

Homework..!

- Practice SoM, PoM
 - Truth table to SoM and PoM
 - When given SoM or PoM, construct the truth table
 - Conversion of SoM to PoM and PoM to SoM
- Be familiar with De Morgan's Theorem
- Circuit construction
 - Construct the circuit when the expression is given
 - Construct the expression when the circuit is given

Thank You..!

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Boolean Algebraic Expression Simplification

Boolean Algebra for Simplification

- Boolean Algebra is a mathematical branch.
- It has its own set of Axioms and laws.
- An expression can be manipulated using rules, laws and theorems.
- It can be used to simplify Boolean expressions.

Boolean Algebra - Postulates

- *Postulate (Axiom)* -

A thing suggested or assumed as true as the basis for mathematical reasoning.

$0.0 = 0$	$0+1 = 1$
$0.1 = 0$	$1+0 = 1$
$1.0 = 0$	$1+1 = 1$
$1.1 = 1$	$\overline{0} = 1$
$0+0 = 0$	$\overline{1} = 0$

Boolean Algebra – Laws

- Annulment Law

- $A \cdot 0 = 0$

- $A + 1 = 1$

- Identity Law

- $A + 0 = A$

- $A \cdot 1 = A$

- Idempotent Law

- $A + A = A$

- $A \cdot A = A$

- Complement Law

- $A \cdot \bar{A} = 0$

- $A + \bar{A} = 1$

- Commutative Law

- $A \cdot B = B \cdot A$

- $A + B = B + A$

- Double Negation Law

- $\overline{\bar{A}} = A$

Boolean Algebra – Laws (Cont.)

- Distributive Law

- $A(B+C) = A.B + A.C$

- Identity Law

- $A + A = A$

- $A . A = A$

- Redundancy Law

- $A + A.B = A$

- $A (A+B) = A$

- Law

- $A + \bar{A}.B = A + B$

- $A(\bar{A} + B) = A.B$

- Law

- $A.B + \bar{A}.B = B$

- $(A + B).(\bar{A} + B) = B$

-

Boolean Algebra - Theorems

- De Morgan's Theorem
 - $\overline{(A \cdot B)} = \bar{A} + \bar{B}$
 - $\overline{(A + B)} = \bar{A} \cdot \bar{B}$

Simplification

- Simple expressions are always with less number of gates.
- A boolean function can be presented in SoM or PoM canonical forms.
- If the output column has more 1s than 0s then presenting the expression in PoM canonical form would be cost effective.
- Otherwise the expression can be presented in SoM.
- However, there are ways to further simplify the expression.

Boolean Algebraic Simplification

- Given expression can be further simplified using boolean algebraic laws and theorems.

- Ex. $F = (x + \bar{y} + \bar{z}).(x + \bar{y}.z)$
$$\begin{aligned} &= x.x + x.\bar{y}.z + x.\bar{y} + \bar{y}.\bar{y}.z + x.\bar{z} + \bar{y}.z.\bar{z} \\ &= x(1 + \bar{y}.z + \bar{y} + \bar{z}) + \bar{y}.z \\ &= x + \bar{y}.z \end{aligned}$$

Exercise..!

- Simplify the following boolean expression

$$F(x, y, z) = x.\bar{y} + x.\bar{z} + y.\bar{z} + x.y.z + y.z$$

Answer

- $= x.\bar{y} + x.\bar{z} + y.\bar{z} + x.y.z + y.z$
- $= x.\bar{y} + x.\bar{z} + x.y.z + y(\bar{z} + z)$
- $= x.\bar{y} + x.\bar{z} + x.y.z + y$
- $= x.\bar{y} + x.\bar{z} + y(x.y + 1)$
- $= x.\bar{y} + x.\bar{z} + y$
- $= (x.\bar{y} + y) + x.\bar{z}$
- $= (x + y) + x.\bar{z}$
- $= (x + x.\bar{z}) + y$
- $= x + y$

Exercise..!

- Simplify the following boolean expression using De Morgan's Theorem

$$F = \overline{(x.y + \bar{y}.z) + (x.z + \bar{x}.\bar{z})}$$

Answer

- $= \overline{(x.y + \bar{y}.z)} + \overline{(x.z + \bar{x}.\bar{z})}$
- $= \overline{(x.y + \bar{y}.z)} . \overline{(x.z + \bar{x}.\bar{z})}$
- $= (\overline{x.y} . \overline{\bar{y}.z}) . (\overline{x.z} . \overline{\bar{x}.\bar{z}})$
- $= ((\bar{x} + \bar{y}) . (y + \bar{z})) . ((\bar{x} + \bar{z}) . (x + z))$
- $= (\bar{x}.y + \bar{x}.\bar{z} + \bar{y}.y + \bar{y}.\bar{z}) . (\bar{x}.x + \bar{x}.z + \bar{z}.x + \bar{z}.z)$
- $= (\bar{x}.y + \bar{x}.\bar{z} + \bar{y}.\bar{z}) . (\bar{x}.z + \bar{z}.x)$
- $= \bar{x}.y.z + \bar{x}.y.\bar{z}.x + \bar{x}.\bar{z}.z + \bar{x}.\bar{z}.x + \bar{y}.\bar{z}.\bar{x}.z + \bar{y}.\bar{z}.x$
- $= \bar{x}.y.z + x.\bar{y}.\bar{z}$

Next...

Karnaugh Map

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Karnaugh Map

Karnaugh Map (K-Map)

- K-Map is a gate level minimization technique.
- A truth table is represented in an alternative diagram which is made up of cells.
- Each cell represents a Minterm in the truth table.
- A set of well defined rules to be applied for simplification.
- If rules are applied properly, it guarantees for the generation of simplest expression.

Karnaugh Map - Representation

- If a boolean function has n variables, then its truth table will have 2^n number of rows.
- Similarly its K-Map will have 2^n number of cells.
- Ex. 3 variable K-Map

Each cell is for a *minterm*.

	$x = 0$ $y = 0$	$x = 0$ $y = 1$	$x = 1$ $y = 1$	$x = 1$ $y = 0$
$z = 0$				
$z = 1$				

Karnaugh Map - Characteristics

- Only one variable can be changed when considering two adjacent columns or rows.

	$x = 0$ $y = 0$	$x = 0$ $y = 1$	$x = 1$ $y = 1$	$x = 1$ $y = 0$
$z = 0$				
$z = 1$				

K-Map: Simplification

- For the simplification we group cell content and find unchanged variables within groups.

$$F = \bar{x}.\bar{y}.z + \bar{x}.y.z + x.\bar{y}.\bar{z} + x.y.\bar{z}$$

	$x = 0$ $y = 0$	$x = 0$ $y = 1$	$x = 1$ $y = 1$	$x = 1$ $y = 0$
$z = 0$				
$z = 1$				

K-Map: Simplification (Cont.)

- For the simplification we group cell content and find unchanged variables within groups.

$$F = \bar{x}.\bar{y}.z + \bar{x}.y.z + x.\bar{y}.\bar{z} + x.y.\bar{z}$$

Mapping

	$x = 0$ $y = 0$	$x = 0$ $y = 1$	$x = 1$ $y = 1$	$x = 1$ $y = 0$
$z = 0$	0	0	1	1
$z = 1$	1	1	0	0

K-Map: Simplification (Cont.)

- For the simplification we group cell content and find unchanged variables within groups.

$$F = \bar{x}.\bar{y}.z + \bar{x}.y.z + x.\bar{y}.\bar{z} + x.y.\bar{z}$$

Grouping

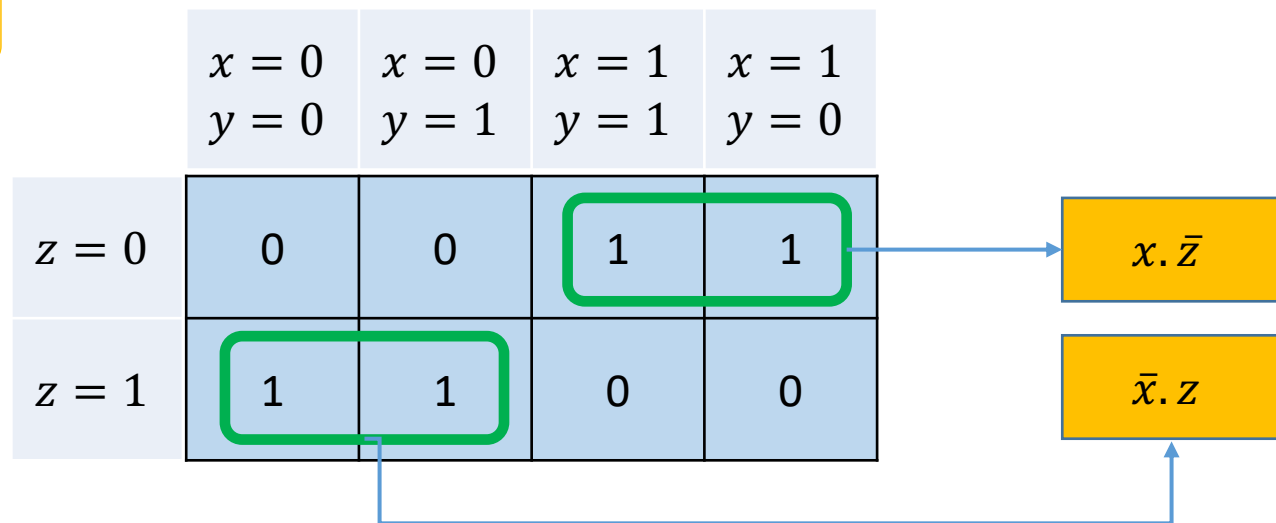
	$x = 0$ $y = 0$	$x = 0$ $y = 1$	$x = 1$ $y = 1$	$x = 1$ $y = 0$
$z = 0$	0	0	1	1
$z = 1$	1	1	0	0

K-Map: Simplification (Cont.)

- For the simplification we group cell content and find unchanged variables within groups.

$$F = \bar{x}.\bar{y}.z + \bar{x}.y.z + x.\bar{y}.\bar{z} + x.y.\bar{z}$$

Deriving

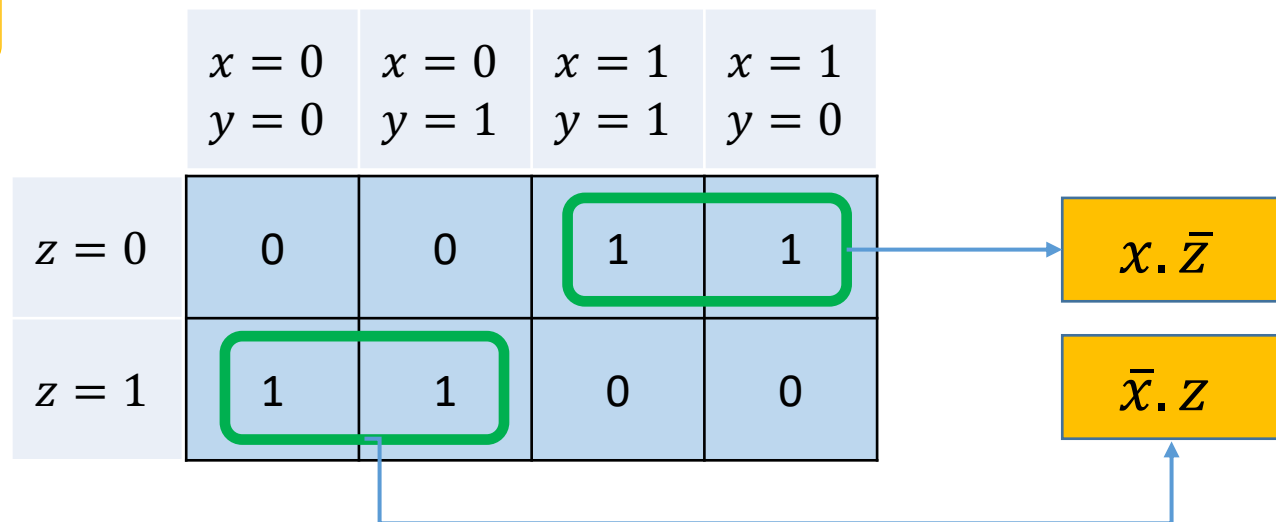


K-Map: Simplification (Cont.)

- For the simplification we group cell content and find unchanged variables within groups.

$$F = \bar{x}.\bar{y}.z + \bar{x}.y.z + x.\bar{y}.\bar{z} + x.y.\bar{z} = x.\bar{z} + \bar{x}.z$$

Deriving



SoP and PoS Expressions

- If the simplified expression is needed in the form of Sum of Products, then 1s are grouped.
- If the simplified expression is needed in the form of Product of Sums, then 0s are grouped.
 - Similar to deriving the expression from the truth table.

Next...

K-Map Grouping Rules

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

K-Map Grouping Rules

K-Map Rules [1]

- Groups should contain only one type
 - Groups should contain only 1s if the expression is required in the form of Sum of Products.
 - Groups should contain only 0s if the expression is required in the form of Product of Sums.

$y \backslash x$	0	1
0	1	0
1	1	0

Correct

$y \backslash x$	0	1
0	1	0
1	1	0

Incorrect

K-Map Rules [2]

- Groups should be formed vertical or horizontal.
 - Expansion should be done in vertically or horizontally.
- Diagonal groups are not allowed.

$y \backslash x$	0	1
0	1	0
1	1	0

Correct

$y \backslash x$	0	1
0	0	1
1	1	0

Incorrect

K-Map Rules [3]

- Groups can only cover 2^n number of cells where $n \geq 0$.

ab \ cd	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	1	0
10	0	0	0	0

Correct

ab \ cd	00	01	11	10
00	1	1	1	0
01	0	0	0	0
11	1	1	1	1
10	1	0	0	0

Incorrect

K-Map Rules [4]

- Each group should be in its maximum size.

ab \ cd	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	1	1	0	0
10	1	1	0	0

Correct

ab \ cd	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	1	1	0	
10	1	1	0	0

Incorrect

K-Map Rules [4.1]

- Groups must be overlapped if it maximizes the groups' sizes.

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	0	0
11	0	0	0	0
10	0	0	0	0

Correct

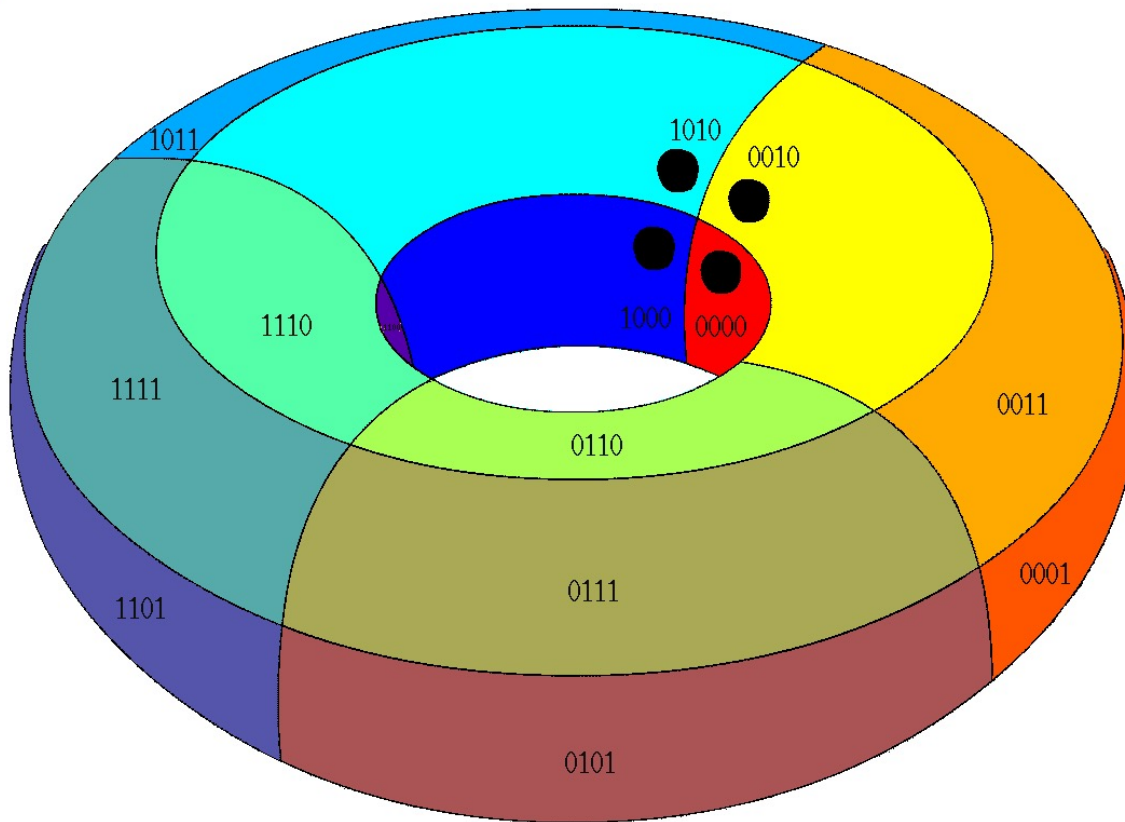
ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	0	0
11	0	0	0	0
10	0	0	0	0

Incorrect

K-Map Rules [4.2]

- Group must be formed or overlapped by considering the uppermost row and the lowermost row are adjacent.
- Similarly it is assumed that the leftmost column and the rightmost column are adjacent.
- Groups may formed around the table.

K-Map Rules [4.2] (Cont.)



K-Map Rules [4.2] (Cont.)

- Wrapping the groups around the table.

ab \ cd	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	0	0	0
10	1	1	0	1

Correct

ab \ cd	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	0	0	0
10	1	1	0	1

Incorrect

K-Map Rules [4.3]

- There should be as few groups as possible while maximizing the sizes of groups.

ab \ cd	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	1	1	0	0
10	1	1	0	0

Correct

ab \ cd	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	1	1	0	0
10	1	1	0	0

Incorrect

Next...

Exercises on K-Map

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Exercises on K-Map

Exercise..

- Draw the K-Map for the following Boolean function and derive the simplified expression in Standard Sum of Products form by applying grouping rules.
- $F = \bar{a}.\bar{b}.\bar{c}.\bar{d} + \bar{a}.\bar{b}.c.\bar{d} + \bar{a}.b.\bar{c}.\bar{d} + \bar{a}.b.c.\bar{d} + a.\bar{b}.\bar{c}.\bar{d} + a.\bar{b}.c.\bar{d}$

Answers (K-Map)

- K-Map for the function F,
- $F = \bar{a}.\bar{b}.\bar{c}.\bar{d} + \bar{a}.\bar{b}.c.\bar{d} + \bar{a}.b.\bar{c}.\bar{d} + \bar{a}.b.c.\bar{d} + a.\bar{b}.\bar{c}.\bar{d} + a.\bar{b}.c.\bar{d}$

cd \ ab	ab			
	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	0	0	0
10	1	1	0	1

Answers (Grouping)

- Group 1 (Blue)
- Unchanged: $\bar{a}$ and $\bar{d}$
- Simplified term: $\bar{a} \cdot \bar{d}$

cd \ ab	ab			
	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	0	0	0
10	1	1	0	1

Answers (Grouping)

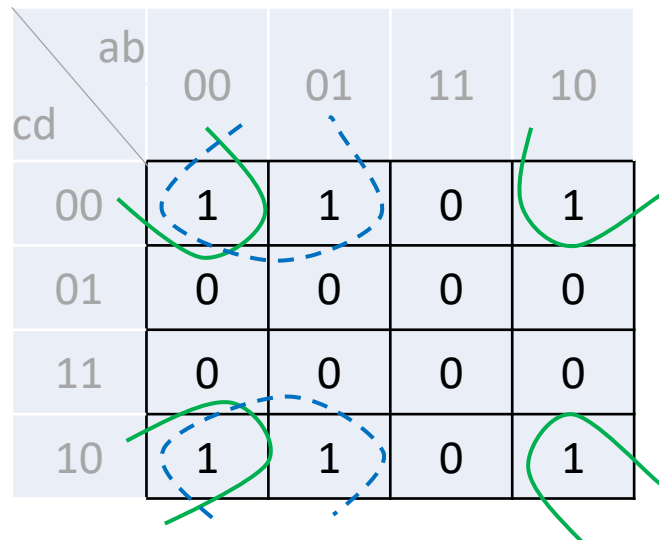
- Group 2 (Green)
- Unchanged: $\bar{b}$ and $\bar{d}$
- Simplified term: $\bar{b} \cdot \bar{d}$

cd \ ab	ab			
	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	0	0	0
10	1	1	0	1

Answers (Grouping)

- Simplified expression: $\bar{a} \cdot \bar{d} + \bar{b} \cdot \bar{d}$
- Original expression: $\bar{a} \cdot \bar{b} \cdot \bar{c} \cdot \bar{d} + \bar{a} \cdot \bar{b} \cdot c \cdot \bar{d} + \bar{a} \cdot b \cdot \bar{c} \cdot \bar{d} + \bar{a} \cdot b \cdot c \cdot \bar{d} + a \cdot \bar{b} \cdot \bar{c} \cdot \bar{d} + a \cdot \bar{b} \cdot c \cdot \bar{d}$

ab \ cd	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	0	0	0
10	1	1	0	1



Answers (Standard PoS)

- What if we want to derive the simplified expression in Standard Product of Sums form?

		ab			
		00	01	11	10
cd	00	1	1	0	1
	01	0	0	0	0
	11	0	0	0	0
	10	1	1	0	1

Answers (Standard PoS)

- If we want to derive the simplified expression in Standard Product of Sums form,
 - Group 0s by following grouping rules

		ab			
		00	01	11	10
cd	00	1	1	0	1
	01	0	0	0	0
	11	0	0	0	0
	10	1	1	0	1

Answers (Standard PoS)

- Simplified expression would be F' ,
 - $F' = (\bar{a} + \bar{b}) \cdot (\bar{d})$

cd \ ab	ab			
	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	0	0	0
10	1	1	0	1

Exercise..

- Derive the simplified Boolean algebraic expression for the following K-Map in
 - Standard Sum of Products form
 - Standard Product of Sums form

cd \ ab				
	00	01	11	10
00	1	0	0	1
01	0	1	1	0
11	0	0	1	0
10	1	0	0	1

Answers

- Derive the simplified Boolean algebraic expression for the following K-Map in
 - Standard Sum of Products form

ab \ cd		ab			
		00	01	11	10
cd	00	1	0	0	1
	01	0	1	1	0
	11	0	0	1	0
	10	1	0	0	1

Answers

- Derive the simplified Boolean algebraic expression for the following K-Map in
 - Standard Product of Sums form

ab \ cd		ab			
		00	01	11	10
cd	00	1	0	0	1
	01	0	1	1	0
	11	0	0	1	0
	10	1	0	0	1

Don't Care Condition - x

- There are functions that output is not defined for its input patterns.
- These are called,
 - Incompletely specified functions
 - Incompletely Defined Functions
- These undefined/ unspecified input patterns are called Don't Care Conditions.
- We can exploit these don't care conditions in K-Map simplifications.

Don't Care - Example

- Let's Assume function F is defined as,

$$F(a, b, c, d) = \sum (1, 3, 5, 7, 9)$$

- F's don't care conditions are defined as,

$$G(a, b, c, d) = \sum (11, 13)$$

Don't Care – Example (Mapping)

$$F(a, b, c, d) = \sum (1, 3, 5, 7, 9)$$

$$G(a, b, c, d) = \sum (11, 13)$$

ab \ cd	00	01	11	10
00	0	0	0	0
01	1	1	X	1
11	1	1	0	X
10	0	0	0	0

Don't Care - Example

$$F(a, b, c, d) = \sum (1, 3, 5, 7, 9)$$

$$G(a, b, c, d) = \sum (11, 13)$$

cd \ ab	ab			
	00	01	11	10
00	0	0	0	0
01	1	1	X	1
11	1	1	0	X
10	0	0	0	0

Don't Care – Suboptimal Grouping

$$F(a, b, c, d) = \sum (1, 3, 5, 7, 9)$$

$$G(a, b, c, d) = \sum (11, 13)$$

ab \ cd	00	01	11	10
00	0	0	0	0
01	1	1	X	1
11	1	1	0	X
10	0	0	0	0

Don't Care – Optimal Grouping #1

$$F(a, b, c, d) = \sum (1, 3, 5, 7, 9)$$

$$G(a, b, c, d) = \sum (11, 13)$$

ab \ cd	00	01	11	10
00	0	0	0	0
01	1	1	X	1
11	1	1	0	X
10	0	0	0	0

$$\bar{c}d + \bar{a}d$$

Don't Care – Optimal Grouping #2

$$F(a, b, c, d) = \sum (1, 3, 5, 7, 9)$$

$$G(a, b, c, d) = \sum (11, 13)$$

ab \ cd	00	01	11	10
00	0	0	0	0
01	1	1	X	1
11	1	1	0	X
10	0	0	0	0

$$\bar{b}d + \bar{a}d$$

Next...

How K-Map works [OPTIONAL]

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

How K-Map works

[OPTIONAL]

Digging It Deep: K-Map

How does it work?

How does K-Map
produce the simplest
expression?



Questions to be Asked...

- Why do we group?
- Why groups are in 2^n ?
- Why groups are formed vertical or horizontal?
- Why do we flip only one variable between two adjacent rows or columns?

Explanation

- K-Map ensures that a group recognizes a set of terms that has common variable states.

c \ ab	ab			
	00	01	11	10
0	0	1	0	0
1	0	1	0	0

- Grouped terms: $\bar{a}.b.\bar{c} + \bar{a}.b.c$
- Here, both terms have a common state ($\bar{a}.b$) for variables a and b .
- The particular state is common for all possible states of the other variable.

Explanation (Cont.)

- When we have a common state for a set of variables over all the possible states of another variable, then the latter variable is irrelevant to the expression.

$$\bar{a}.b.\bar{c} + \bar{a}.b.c = \bar{a}.b$$

- By having a group of 2 cells we can recognize a single irrelevant variable (2 cells cover all possible states for a single variable).
- If we can group 4 cells then we can recognize two irrelevant variables (4 cells cover all possible states for two variables).
- This is why we form groups with 2^n number of cells.
- We are trying to eliminate variables as much as possible to make the expression simple.

Explanation (Cont.)

- Forming groups vertically or horizontally, ensures covering all the possible states for a given set of variables.

- Green Group Terms:

$$\bar{a}.b.\bar{c} + \bar{a}.b.c$$

Common state ($\bar{a}.b$) for all possible states of variable c .

c \ ab	ab			
	00	01	11	10
0	0	1	0	1
1	0	1	1	0

- Red Group: $a.\bar{b}.\bar{c} + a.b.c$

Common state (a) doesn't appear with all possible states of variable b and c .

- Restriction on flipping 1 variable at a time also do the same.

Next...

Single Gate Type Circuits

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Single Gate Type Circuits

Single Gate Type Circuits

- Circuits are preferred to be constructed in a single gate type.
- Manufacturing ICs with different types of gates are expensive.
- There can be unused gates in ICs if those ICs are manufactured with different types of gates.

Functional Completeness

- AND, OR and NOT operations are the primary functionalities.
- Therefore, any set of gates that can demonstrate all three functionalities is called *Functionally Complete Set*.
- Any Boolean expression can be constructed using a functionally complete set of gates.
- By its definition, {AND, OR, NOT} is a functionally complete set.

Functional Completeness

A functionally complete set of logical connectives or Boolean operators is one which can be used to express all possible truth tables by combining members of the set into a Boolean expression.
~ Wikipedia

{AND, NOT}

- Is the set {AND, NOT} a functionally complete set?
- The missing primary functionality is OR.
- Then {AND, NOT} should be able to demonstrate OR.

$$\begin{aligned} A + B &= \overline{\overline{A + B}} \\ A + B &= \overline{\overline{A} \cdot \overline{B}} \end{aligned}$$

- OR operation can be performed by AND and NOT.
- Therefore, {AND, NOT} is functionally complete.

{OR, NOT}

- Similarly, {OR, NOT} is a functionally complete set.
- Here, missing operation is AND
- Then {OR, NOT} should be able to demonstrate AND.

$$\begin{aligned} A \cdot B &= \overline{\overline{A \cdot B}} \\ A \cdot B &= \overline{\overline{A} + \overline{B}} \end{aligned}$$

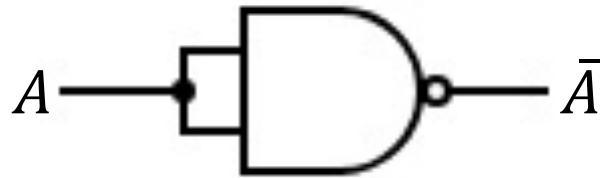
- AND operation can be performed by OR and NOT.
- Therefore, {OR, NOT} is functionally complete.

{NAND} , {NOR}

- NAND and NOR gates are identified as individually functionally complete.
- Thus, digital circuits can be implemented by single gate type (NAND or NOR).
- ICs come with several gates from single gate type.

NOT from NAND

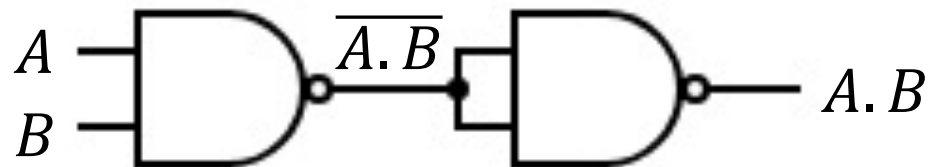
- NAND gate can be used as an inverter.



$$\overline{A \cdot A} = \bar{A}$$

AND from NAND

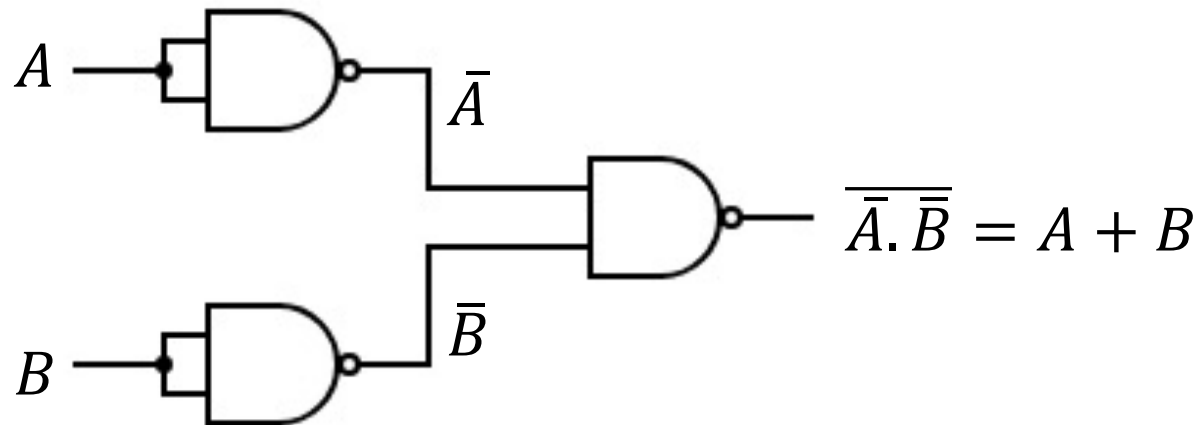
- A set of NAND gates can be used as an AND gate.



OR from NAND

- A set of NAND gates can be used as an OR gate.

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$
$$A + B = \overline{\overline{A} \cdot \overline{B}}$$



NAND as an Universal Gate

- NAND can implement XOR, NOR, XNOR.
- NAND can implement any Boolean function without any other gate type (Universal).
- NOR is also a Universal Gate.

Exercise..

How can we use NOR gate,

- to implement NOT gate?
- to implement AND gate?
- to implement OR gate?



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

NOT from NOR

- NOR gate can be used as an inverter.

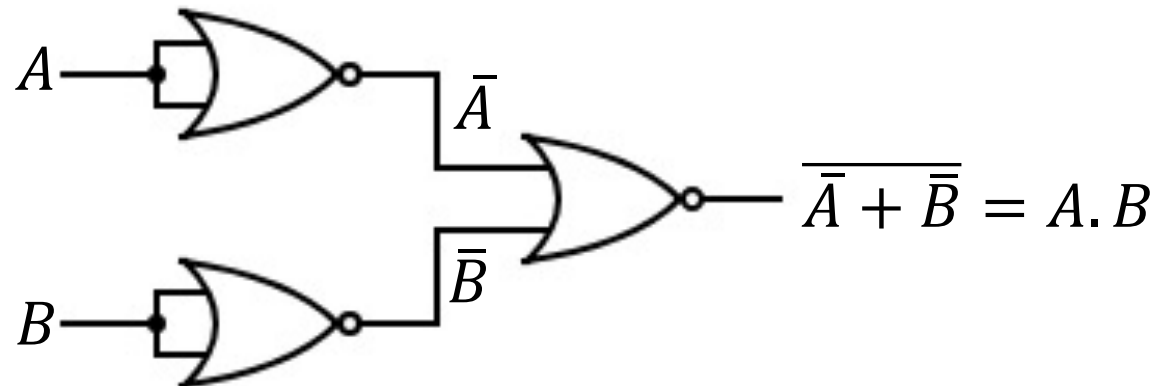


$$\overline{A + A} = \bar{A}$$

AND from NOR

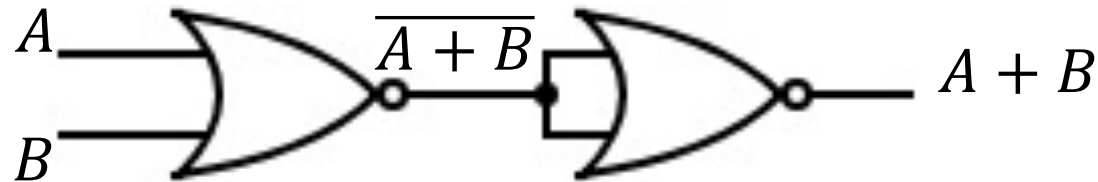
- A set of NOR gates can be used to perform AND operation.

$$A.B = \overline{\overline{A.B}}$$
$$A.B = \overline{\overline{A} + \overline{B}}$$



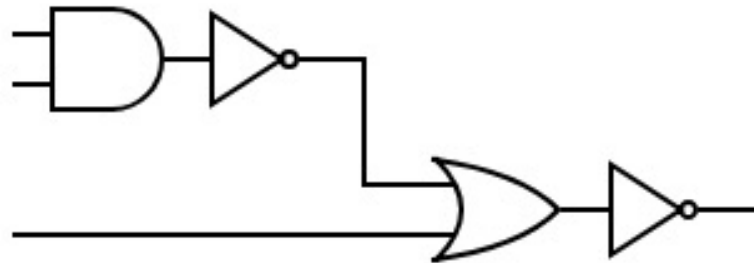
OR from NOR

- A set of NOR gates can be used to perform OR operation.



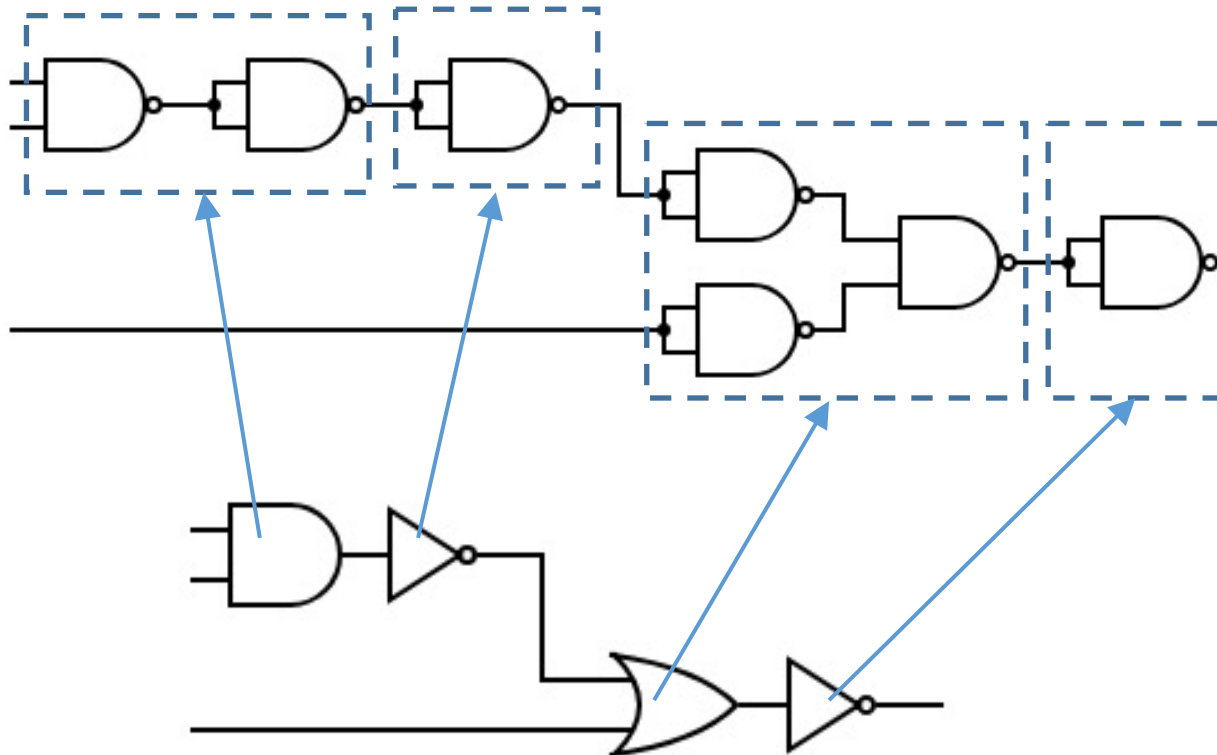
Exercise..

- Following circuit shows the gate arrangement for the boolean expression $F = \overline{\overline{x} \cdot \overline{y}} + z$
- Derive a single gate type circuit for it by substituting NAND gates.



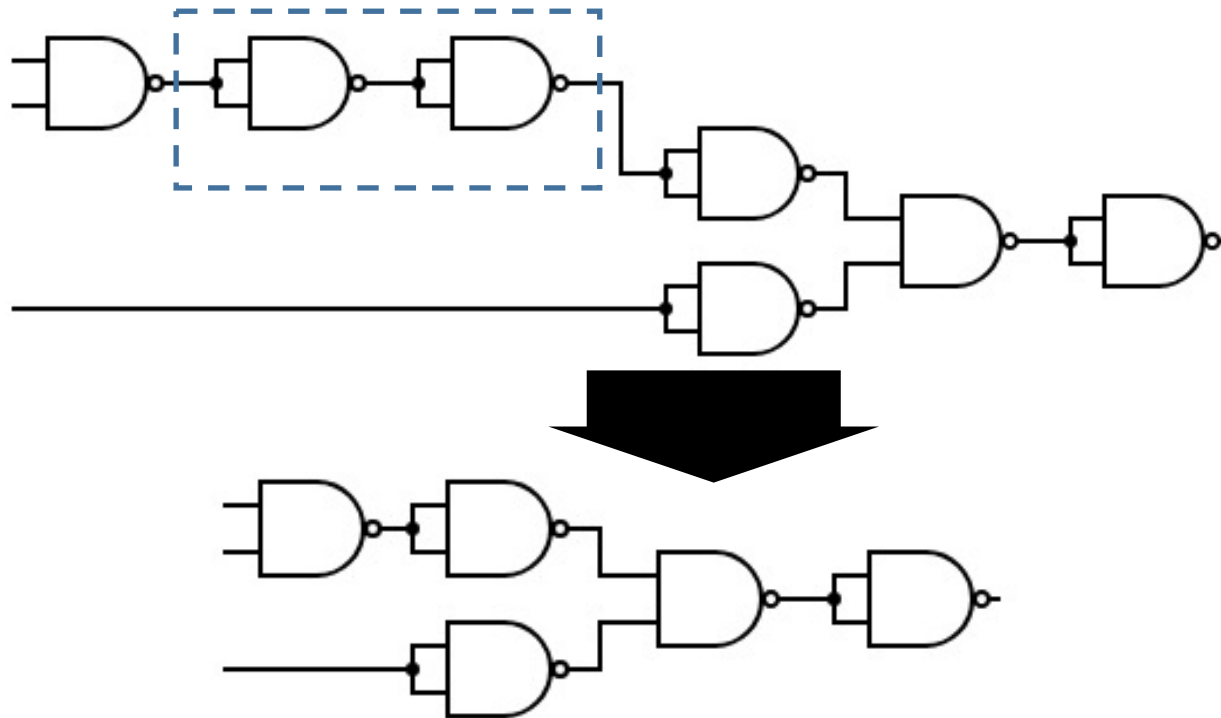
Answer

- Direct Substitution would construct a circuit as follows:



Answer

- Double negations can be removed and the circuit can be simplified further:



Thank You..!
